

Prazo: 02/07/2026 até as 12h15 (2 pontos) - até as 16h (1 ponto)

Entrega: e-mail para bidu@ci.ufpb.br com assunto exato: [POO-261-TE002] Nome - Matrícula

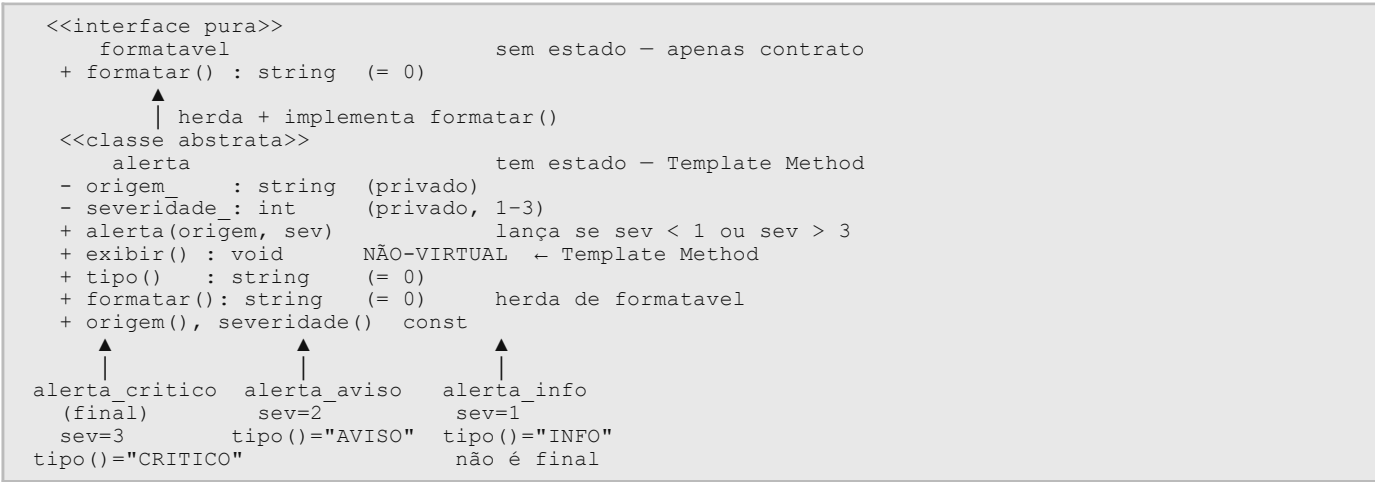
Anexo: ZIP do repositório Git ou URL pública do GitHub

Ferramentas: LLMs são permitidos — siga o Guia de Uso Inteligente de IA já distribuído

Contexto

Você irá implementar um sistema de monitoramento que gera, registra e exibe alertas de diferentes tipos. O exercício integra todos os conceitos vistos nas aulas: interface pura, classe abstrata, herança, override, final, padrão Template Method e polimorfismo dinâmico via `vector<unique_ptr<Base>>`.

Diagrama de classes



Especificação das classes

1. **formatavel** — interface pura
- Método virtual puro: `string formatar() const = 0`
 - Destrutor virtual = default.
 - Sem atributos, sem implementação além do destrutor.

2. **alerta** — classe abstrata (herda `formatavel`):

Membro	Tipo / assinatura	Comportamento
<code>origem_</code>	<code>string</code> (privado)	Identificador da fonte do alerta (ex: "sensor1")
<code>severidade_</code>	<code>int</code> (privado)	Nível: 1 = info, 2 = aviso, 3 = crítico
<code>alerta(origem, sev)</code>	construtor público	Inicializa os dois atributos; lança <code>std::invalid_argument</code> se <code>sev < 1</code> ou <code>sev > 3</code>
<code>exibir() const</code>	<code>void</code> — NÃO-VIRTUAL	Template Method: imprime "[<code>tipo()</code>] <code>origem_</code> : <code>formatar()</code> "
<code>tipo() const</code>	<code>string</code> — virtual puro	Retorna o tipo como <code>string</code> ("CRITICO", "AVISO", "INFO")
<code>formatar() const</code>	<code>string</code> — virtual puro (de <code>formatavel</code>)	Retorna a mensagem formatada do alerta
<code>origem() const</code>	<code>string</code> — getter const	Retorna <code>origem_</code>
<code>severidade() const</code>	<code>int</code> — getter const	Retorna <code>severidade_</code>

O método **exibir()** deve ser implementado em `alerta` da seguinte forma:

```
void alerta::exibir() const {
    std::cout << "[" << tipo() // chama o virtual tipo()
```

```

        << "]" << origem_ << ": "
        << formatar() // chama o virtual formatar()
        << "\n";
    }

```

3. alerta_critico final : public alerta

- Atributo privado adicional: `mensagem_` (string).
- Construtor: `alerta_critico(origem, mensagem)` — chama `alerta(origem, 3)` (severidade sempre 3).
- `tipo()` override → retorna "CRITICO".
- `formatar()` override → retorna `mensagem_`.
- Classe marcada como `final`: não pode ser estendida.

4. alerta_aviso : public alerta

- Atributo privado adicional: `mensagem_` (string).
- Construtor chama `alerta(origem, 2)`.
- `tipo()` override → "AVISO". `formatar()` override → `mensagem_`.

5. alerta_info : public alerta

- Atributo privado adicional: `mensagem_` (string).
- Construtor chama `alerta(origem, 1)`.
- `tipo()` override → "INFO". `formatar()` override → `mensagem_`.
- NÃO é `final` — pode ser especializada (veja desafio).

6. central — gerencia o vetor de alertas:

```

class central {
    std::vector<std::unique_ptr<alerta>> alertas_;
public:
    void registrar(std::unique_ptr<alerta> a); // recebe por valor (move)
    void exibir_todos() const; // itera e chama a->exibir()
    int total() const noexcept; // retorna alertas_.size()
};

```

main() e saída esperada

```

#include "alerta.hpp"

int main() {
    central c;

    c.registrar(std::make_unique<alerta_critico>(
        "sensor1", "temperatura 95.3 graus"));
    c.registrar(std::make_unique<alerta_aviso>(
        "sensor2", "umidade 78.5 pct"));
    c.registrar(std::make_unique<alerta_info>(
        "sistema", "reinício agendado"));

    std::cout << "Total: " << c.total() << " alertas\n\n";
    c.exibir_todos();
}

```

Saída esperada:

```

Total: 3 alertas

[CRITICO] sensor1: temperatura 95.3 graus
[AVISO] sensor2: umidade 78.5 pct
[INFO] sistema: reinicio agendado

```

Estrutura de arquivos sugerida

```

central-alertas/
├── CMakeLists.txt
├── alerta.hpp      ← formatavel + alerta (abstrata)
├── alerta.cpp      ← implementações de alerta::exibir() e construtor
├── alertas/
│   ├── alerta_critico.hpp / .cpp
│   ├── alerta_aviso.hpp  / .cpp
│   └── alerta_info.hpp    / .cpp

```

```
└─ central.hpp / .cpp
   main.cpp
```

Conceitos cobertos

Conceito	Onde aparece neste exercício
Interface pura (Aula 17)	formatavel: todos os métodos = 0, sem estado
Classe abstrata (Aula 17)	alerta: tem estado + Template Method + métodos puros
Herança simples (Aula 15)	alerta_critico / alerta_aviso / alerta_info → alerta
override (Aula 17)	tipo() e formatar() em todas as derivadas
final em classe (Aula 17)	class alerta_critico final : public alerta
Destrutor virtual (Aula 15)	alerta herda formatavel — ambas têm virtual ~
Template Method (Aula 17)	alerta::exibir() — não-virtual, chama tipo() e formatar()
Polimorfismo dinâmico (Aula 18)	central::exibir_todos() itera vector<unique_ptr<alerta>>

Checklist de autoavaliação

	Critério de autoavaliação
☐	formatavel tem apenas métodos puros e destrutor virtual — sem estado nem implementação
☐	alerta::exibir() é não-virtual e chama tipo() e formatar() via despacho dinâmico
☐	alerta lança std::invalid_argument quando severidade estiver fora de [1, 3]
☐	alerta_critico está marcado como final; severidade é sempre 3 (hardcoded no construtor)
☐	alerta_aviso e alerta_info usam override em tipo() e formatar()
☐	central::registrar() recebe unique_ptr por valor e usa std::move ao adicionar ao vetor
☐	central::exibir_todos() itera com const auto& e chama exibir() via ponteiro base
☐	main() usa make_unique — sem new/delete crus em lugar nenhum
☐	Compila sem warnings com -Wall -Wextra -std=c++17
☐	Nenhum erro reportado por -fsanitize=address,undefined

Desafio (se sobrar tempo): adicione `alerta_debug : public alerta_info` — demonstrando que `alerta_info` não é final (mas `alerta_critico` é).

`alerta_debug` recebe uma mensagem adicional de contexto técnico e a appende ao `formatar()`. Registre um `alerta_debug` na `central` e verifique que `exibir()` imprime "[INFO]" com a mensagem formatada.